



Il Ritorno delle Pizze (Versione Easy) (fattorino_easy)

È l'ora di pranzo allo stage per le Olimpiadi di Informatica e tutti gli studenti sono affamati! Tommaso aveva promesso di preparare la pizza per tutti, ma come al solito è troppo lento. Samuele, stufo di aspettare, decide di andare lui stesso in pizzeria a prendere le pizze.

Via Gemona è composta da N luoghi in fila, numerati da 1 a N . Samuele impiega t_i minuti per camminare dal luogo i al luogo $i + 1$, e viceversa.

Samuele intanto si è perso in via Gemona e, dato che si è fatto tardi, decide di tornare al Toppo, in posizione T .

In K luoghi $C_1, \dots, C_K$ di via Gemona ci sono dei chioschi che vendono tranci di pizza. Il chiosco in posizione C_i vende un trancio di pizza con un livello di soddisfazione S_i (misurato in «quanto vale la pena fermarsi»).

Samuele è disposto a fermarsi al massimo in un chiosco lungo il suo percorso verso il Toppo, ma solo se il tempo extra aggiunto al suo percorso è al massimo uguale al livello di soddisfazione della pizza che prenderebbe. In altre parole, se fermarsi a prendere una pizza gli fa perdere t minuti in più rispetto al percorso più veloce, lo farà solo se il livello di soddisfazione è almeno t .

Dato che non sappiamo dove si trova Samuele, determina per ogni possibile posizione iniziale se Samuele si fermerà a prendere la pizza oppure no.

Dati di input

La prima riga contiene gli interi N , K , T , rispettivamente il numero di luoghi, il numero di chioschi e la posizione del Toppo.

La $1 + i$ -esima riga ($1 \leq i \leq N - 1$) contiene un intero t_i , i minuti che ci mette Samuele per andare dal luogo i al luogo $i + 1$, e viceversa.

La $N + i$ -esima riga ($1 \leq i \leq K$) contiene gli interi C_i e S_i , rispettivamente la posizione dell' i -esimo chiosco e il suo livello di soddisfazione.



Tra gli allegati a questo task troverai un template `fattorino_easy.*` con un esempio di implementazione.

Dati di output

Stampa N righe. All' i -esima riga stampa 1 se Samuele, partendo dalla posizione i , si fermerà a prendere la pizza. Stampa 0 altrimenti.

Assunzioni

- $2 \leq N \leq 100000$.
- $1 \leq K \leq N$.
- $1 \leq T \leq N$.
- $1 \leq t_i \leq 10000$ per ogni $1 \leq i \leq N - 1$.
- $1 \leq C_i \leq N$ per ogni $1 \leq i \leq K$.
- $1 \leq S_i \leq 10^9$ per ogni $1 \leq i \leq K$.

Assegnazione del punteggio

Il tuo programma verrà testato su diversi testcase raggruppati in subtask. Per ottenere il punteggio relativo ad un subtask, è necessario risolvere correttamente tutti i test che lo compongono.

- **Subtask 1** (0 punti) Casi d'esempio.
★★★★★
- **Subtask 2** (5 punti) $N \leq 10, K \leq 10$.
★★★★★
- **Subtask 3** (15 punti) $N, K \leq 1000$.
★★★★★★★★★★★★★★★★
- **Subtask 4** (20 punti) $K = 1$.
★★★★★★★★★★★★★★★★
- **Subtask 5** (25 punti) $t_i = 1$ per ogni $1 \leq i \leq N - 1$.
★★★★★★★★★★★★★★★★
- **Subtask 6** (35 punti) Nessuna limitazione aggiuntiva.
★★★★★★★★★★★★★★★★

Esempi di input/output

stdin	stdout
4 5 1 3	1
0 3 10	1
1 0 20	1
3 1 3	1
1 2 5	
3 2 2	
1 7	

Spiegazione

Nel **primo caso d'esempio** ci sono $N = 4$ luoghi, il Toppo è in posizione $T = 4$, e c'è un solo chiosco in posizione 1 con soddisfazione $S = 7$. I tempi di percorrenza sono $t_1 = 3, t_2 = 2, t_3 = 5$.

- Se Samuele parte dalla posizione 1: il chiosco è sul suo percorso diretto $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, non perde tempo extra. Stampa 1.
- Se Samuele parte dalla posizione 2: il percorso ottimale è $2 \rightarrow 3 \rightarrow 4$. Per il chiosco in 1, deve fare $2 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, perdendo $2 \times 3 = 6$ minuti. Dato che $6 \leq 7$, si ferma. Stampa 1.
- Se Samuele parte dalla posizione 3: il percorso ottimale è $3 \rightarrow 4$. Il detour per il chiosco in 1 costa $2 \times (3 + 2) = 10$ minuti. Dato che $10 > 7$, non si ferma. Stampa 0.
- Se Samuele parte dalla posizione 4: è già al Toppo. Il detour per il chiosco in 1 costa $2 \times (3 + 2 + 5) = 20$ minuti. Dato che $20 > 7$, non si ferma. Stampa 0.